Counting Strings Avoiding Forbidden Patterns

Topic.

String algorithms, Aho–Corasick automaton, dynamic programming.

Statement.

Given a dictionary and a number n. Count the number of strings of length n over the alphabet $\{a, b\}$ that do not contain any dictionary word as a substring. The required running time is $O(nL)$, where L is the total length of the dictionary words.

Solution.

Build the Aho–Corasick automaton for the dictionary.

Mark bad vertices: terminal vertices and all vertices from which a terminal vertex is reachable by suffix links. That is, if the automaton gets into a bad vertex, then a dictionary word has already appeared in the string as a substring.

Let

$$dp[v]$$

be the number of good strings of the current length after reading which the automaton is in vertex v.

Let

$$ndp[v]$$

be the same for the next length.

The base is

$$dp[root] = 1,$$

and all other vertices have value 0.

Transition: for each step, iterate over all good vertices v.

If the transition by symbol c from the set $\{a, b\}$ leads to a good vertex u, then

$$ndp[u] += dp[v].$$

That is, for each i from 0 to $n - 1$, we do the following.

For all good vertices v:

$$u = go(v, a).$$

If u is a good vertex, then

$$ndp[u] += dp[v].$$

Also,

$$u = go(v, b).$$

If u is a good vertex, then

$$ndp[u] += dp[v].$$

After that, swap dp and ndp, and set ndp to zero.

The answer is the sum of $dp[v]$ over all good vertices after n steps.

The automaton state after reading a string is uniquely determined by this string. A string is valid if and only if during reading it the automaton never gets into a bad vertex. Therefore, good paths of length n in the automaton correspond to strings of length n that do not contain dictionary words as substrings.

The dynamic programming simply counts the number of such paths.

Complexity.

The number of automaton vertices is

$$O(L).$$

On each of the n layers, for each vertex we make two transitions.

Therefore, the total running time is

$$O(nL).$$

For memory, we store only two dynamic programming layers dp and ndp, each layer of size

$$O(L).$$

The rest of the memory is used by the automaton.